# 作业 _Week4

20244227004+ 蔡博凯

2024 年 10 月 6 日

**第一题** 试述真正例率 (TPR)、假正例率 (FPR) 与查准率 (P)、查全率 (R) 之间的联系。

**解答**

**定义**

真正例率 (TPR) 用于衡量模型对所有正例中正确识别正例的比例。

$$\mathrm{TPR} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}$$

假正例率 (FPR) 衡量的是模型将负例错误地分类为正例的比例。

$$\mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP} + \mathrm{TN}}$$

| 真实情况 | 预测结果 | |
|---|---|---|
| | 正例 | 反例 |
| 正例 | $TP$ (真正例) | $FN$ (假反例) |
| 反例 | $FP$ (假正例) | $TN$ (真反例) |

图 1: 分类结果混淆矩阵

查准率 (P) 衡量的是模型预测为正例的样本中，真正例所占的比例。

$$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}$$

查全率 (R) 等同于真正例率 (TPR)。

$$R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}$$

**联系**

**TPR&FPR**

当提高模型的真正例率时，通常也会导致假正例率上升。原因在于，模型变得“宽松”，以捕捉更多的正例，同时可能会误分类更多的负例为正例。

**P&R**

查准率和查全率通常存在一定的权衡。当模型更加严格时，查准率会提高，但查全率可能会下降，因为模型对正例的识别变得更加保守，漏掉更多的正例（FN 增多）。反之，当查全率提高时，模型可能会更多地预测为正例，从而导致查准率下降（FP 增多）。

**第二题** 若学习器 A 的 F1 值比学习器 B 高，试析 A 的 BEP 值是否也比 B 高。

**解答**

**定义**

F1 值是查准率和查全率的调和平均数，用于权衡模型的精度和召回率之间的平衡。

$$F1 = 2 \times \frac{P \times R}{P + R}$$

BEP 是查准率和查全率相等时的值，代表了模型在查准率和查全率之间达到平衡时的性能表现。

**联系**

虽然 F1 值考虑了查准率和查全率的综合表现，但它并不直接要求查准率和查全率必须相等。F1 值高的模型可能存在查准率和查全率不平衡的情况，只要它们的综合表现较好即可。因此，A 的 F1 值高于 B 只表明 A 的查准率和查全率的综合平均比 B 更高，但并不意味着 A 在 P = R 时的性能一定比 B 好。

**第三题**

试证明logistic函数 $f(\mathbf{x};\beta)=\frac{\mathbf{1}}{\mathbf{1}+\mathbf{e}^{-\beta^{\mathbf{T}}\mathbf{x}}}$ 对于参数$\beta$是非凸的，而如下函数 $L(\beta)=-y_0\beta^T\mathbf{x}+\ln(\mathbf{1}+\mathbf{e}^{\beta^{\mathbf{T}}\mathbf{x}})$ 对于$\beta$是凸的；其中 $y_0$ 为常数。

**解答**

对于多元函数，其 Hessian 矩阵为半正定即为凸函数。

(1)

$$\text{记}\quad y=f(x;\beta)=\frac{1}{1+e^{-\beta^T x}}\qquad \text{对参数}\beta\text{求导}$$

$$\text{一阶导}\quad \frac{\partial y}{\partial \beta}=x(y-y^2)$$

$$\text{二阶导 (hessian 矩阵)}\quad \frac{\partial^2 y}{\partial\beta\partial\beta^T}=xx^Ty(1-y)(1-2y)$$

**分析**

其中 $xx^T$ 秩为 1，非零特征值只有一个，其正负号取决于 $y(1-y)(1-2y)$，由于 $f(x;\beta)$ 值域为 (0,1)，当 y 在 (0,1) 之间变化时，特征值正负号会发生变化，于是 $f(x;\beta)$ 关于 $\beta$ 的 hessian 矩阵非半正定，因此非凸。

(2)

$$L(\beta) = -y_0\beta^T x + ln(1 + e^{\beta^T x}) \quad \text{对参数}\beta\text{求导}$$

$$\text{一阶导} \quad \frac{\partial L}{\partial \beta} = -y_0 x + f(x;\beta)x$$

$$\text{二阶导 (hessian 矩阵)} \quad \frac{\partial^2 L}{\partial \beta \partial \beta^T} = xx^T f(x;\beta)[1 - f(x;\beta)]$$

**分析**

其中 $xx^T$ 秩为 1，非零特征值只有一个，其正负号取决于 $f(x;\beta)[1 - f(x;\beta)]$，由于 $f(x;\beta)$ 值域为 (0,1), 当 $f(x;\beta)$ 在 (0,1) 之间变化时，特征值恒正，于是 $L(\beta)$ 关于 $\beta$ 的 hessian 矩阵半正定，因此是凸的。

**第四题** 用 python 实现对率回归，对西瓜数据集（见表）进行划分。请附程序（含注释）及运行结果（同一 pdf 中）。

| 编号 | 密度 | 含糖率 | 好瓜 |
|---|---|---|---|
| 1 | 0.697 | 0.460 | 是 |
| 2 | 0.774 | 0.376 | 是 |
| 3 | 0.634 | 0.264 | 是 |
| 4 | 0.608 | 0.318 | 是 |
| 5 | 0.556 | 0.215 | 是 |
| 6 | 0.403 | 0.237 | 是 |
| 7 | 0.481 | 0.149 | 是 |
| 8 | 0.437 | 0.211 | 是 |
| 9 | 0.666 | 0.091 | 否 |
| 10 | 0.243 | 0.267 | 否 |
| 11 | 0.245 | 0.057 | 否 |
| 12 | 0.343 | 0.099 | 否 |
| 13 | 0.639 | 0.161 | 否 |
| 14 | 0.657 | 0.198 | 否 |
| 15 | 0.360 | 0.370 | 否 |
| 16 | 0.593 | 0.042 | 否 |
| 17 | 0.719 | 0.103 | 否 |

## 解答

### 概述

我使用 python 实现了一个对率回归模型，使用梯度下降法优化权重。代码包含了计算 sigmoid 函数、损失函数（负对数似然）、梯度计算和模型训练的过程。模型通过拟合数据找到最佳参数，预测结果使用阈值分类（0 或 1），并与 sklearn 的对率回归模型进行比较与可视化。

### 数据集和代码

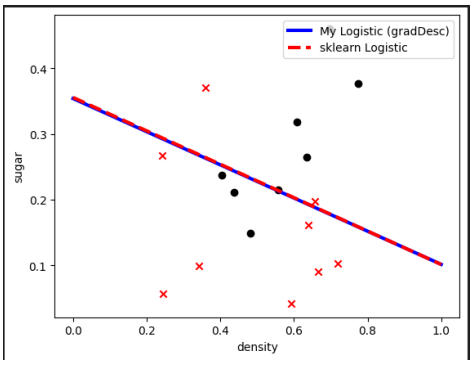
可在附录查看数据集，请参考 西瓜数据集。

可在附录查看完整代码，请参考 对率回归。

### 运行结果

```
0th iteration, cost is 11.881571695897671
100th iteration, cost is 8.954817568096097
200th iteration, cost is 8.744504625734402
300th iteration, cost is 8.70022608572249
400th iteration, cost is 8.688571088177042
500th iteration, cost is 8.685178971227133
600th iteration, cost is 8.684140671980247
700th iteration, cost is 8.683814239534147
800th iteration, cost is 8.683710097134105
900th iteration, cost is 8.683676599903729
```

(a) 训练过程中损失函数变化




(b) 划分结果对比

图 2: 模型运行结果

### 结论

根据图 2(a) 可知，训练过程中损失函数总体呈下降趋势，这说明训练是有效的。根据图 2(b) 可知，此模型基本将两类结果划分出，同时与 sklearn 提供的模型划分效果类似，说明了模型的可靠性。

另外，此模型采用梯度下降来优化，而 sklearn 提供的模型采用拟牛顿法进行优化，运行结果表明二者优化效果类似。

**第五题** 用 python 写一个感知机算法，分别实现 and 和 or。请附程序 (含注释）及运行结果（同一 pdf 中）

**解答**

**概述**

我使用 python 实现了一个简单的感知机类，能够学习 AND 和 OR 逻辑运算。感知机初始化时设定权重、学习率和迭代次数。fit 方法用于训练模型，通过调整权重和偏置项来最小化预测误差。predict 方法根据输入计算加权和，并通过激活函数输出结果。代码最后展示了对 AND 和 OR 操作的训练结果及对应的预测输出。

**代码**

可在附录查看完整代码，请参考 感知机。

**运行结果**

```
Final Outputs:
AND Operation:
Input: [0 0], Predicted Output: 0
Input: [0 1], Predicted Output: 0
Input: [1 0], Predicted Output: 0
Input: [1 1], Predicted Output: 1

OR Operation:
Input: [0 0], Predicted Output: 0
Input: [0 1], Predicted Output: 1
Input: [1 0], Predicted Output: 1
Input: [1 1], Predicted Output: 1
```

图 3: 感知机实现 AND 和 OR

**结论**

可以观察到，感知机正确模拟出了 AND 和 OR。

**第六题**

试证明样本空间中任意点 $\boldsymbol{x}$ 到超平面 $(\boldsymbol{w}, b)$ 的距离为

$$r = \frac{|\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} + b|}{||\boldsymbol{w}||}$$

**解答**

对于任意一点 $\mathbf{x}_0 = (x_1^0, x_2^0, \ldots, x_n^0)$，设其在超平面 $\mathbf{w}^T\mathbf{x} + b = 0$ 上的投影点为 $\mathbf{x}_1 = (x_1^1, x_2^1, \ldots, x_n^1)$，则 $\mathbf{w}^T\mathbf{x}_1 + b = 0$。由于法向量 $\mathbf{w}$ 垂直于超平面，此时向量 $\overrightarrow{x_1x_0}$ 与法向量 $\mathbf{w}$ 平行，因此

$$\mathbf{w} \cdot \overrightarrow{x_1x_0} = \|\|\mathbf{w}\| \cdot \cos\pi \cdot \|\overrightarrow{x_1x_0}\|\| = \|\mathbf{w}\| \cdot \|\overrightarrow{x_1x_0}\| = \|\mathbf{w}\| \cdot r$$

又

$$\begin{aligned}
\mathbf{w} \cdot \overrightarrow{x_1x_0} &= w_1(x_1^0 - x_1^1) + w_2(x_2^0 - x_2^1) + \cdots + w_n(x_n^0 - x_n^1) \\
&= w_1x_1^0 + w_2x_2^0 + \cdots + w_nx_n^0 - (w_1x_1^1 + w_2x_2^1 + \cdots + w_nx_n^1) \\
&= \mathbf{w}^T\mathbf{x}_0 - \mathbf{w}^T\mathbf{x}_1 \\
&= \mathbf{w}^T\mathbf{x}_0 + b
\end{aligned}$$

所以

$$|\mathbf{w}^T\mathbf{x}_0 + b| = \|\mathbf{w}\| \cdot r$$

$$r = \frac{|\mathbf{w}^T\mathbf{x} + b|}{\|\mathbf{w}\|}$$

**第七题**

若整个数据集包含M=100000个示例$\{(x_i, y_i, z_i)\}_{i=1}^{M}$，其中$z_i$为标签。考虑抛物面回归问题$f(x,y) = \frac{x^2}{\omega_1^2} + \frac{y^2}{\omega_2^2}$，试采用批量梯度下降法学习参数，分100个mini-batches、5个epochs训练，并使用均方误差作为损失函数。请写出算法（其中的偏导数请给出具体形式）。

**解答**

1. **初始化参数**：

$$\text{initialize}(\omega_1, \omega_2) \quad \text{learning_rate} = \eta \quad \text{batch_size} = 1000 \quad \text{epochs} = 5$$

2. **定义损失函数**：

$$L(\omega_1, \omega_2, x_i, y_i, z_i) = \left( \frac{x_i^2}{\omega_1^2} + \frac{y_i^2}{\omega_2^2} - z_i \right)^2$$

3. **计算梯度**：

- 对参数 $\omega_1$ 的偏导数：

$$\frac{\partial L}{\partial \omega_1} = 2 \left( \frac{x_i^2}{\omega_1^2} + \frac{y_i^2}{\omega_2^2} - z_i \right) \cdot \left( -\frac{2x_i^2}{\omega_1^3} \right) = -\frac{4x_i^2 \left( \frac{x_i^2}{\omega_1^2} + \frac{y_i^2}{\omega_2^2} - z_i \right)}{\omega_1^3}$$

- 对参数 $\omega_2$ 的偏导数：

$$\frac{\partial L}{\partial \omega_2} = 2 \left( \frac{x_i^2}{\omega_1^2} + \frac{y_i^2}{\omega_2^2} - z_i \right) \cdot \left( -\frac{2y_i^2}{\omega_2^3} \right) = -\frac{4y_i^2 \left( \frac{x_i^2}{\omega_1^2} + \frac{y_i^2}{\omega_2^2} - z_i \right)}{\omega_2^3}$$

4. **小批量梯度下降**：

- 对于每个训练轮数（epoch）：

- - 随机打乱数据集 $D$。

- - - - 对于数据集中的每个批次：

$$\text{grad}_{\omega_1}^{\text{sum}} = \sum_{(x_i, y_i, z_i) \in \text{batch}} \frac{\partial L}{\partial \omega_1}$$

$$\text{grad}_{\omega_2}^{\text{sum}} = \sum_{(x_i, y_i, z_i) \in \text{batch}} \frac{\partial L}{\partial \omega_2}$$

- - - - 更新参数：

$$\omega_1 \leftarrow \omega_1 - \eta \cdot \frac{\text{grad}_{\omega_1}^{\text{sum}}}{\text{batch_size}}$$

$$\omega_2 \leftarrow \omega_2 - \eta \cdot \frac{\text{grad}_{\omega_2}^{\text{sum}}}{\text{batch_size}}$$

# 参考文献

[1] https://github.com/hanmq/MachineLearning_Zhouzhihua_ProblemSets

[2] https://datawhalechina.github.io/pumpkin-book

[3] 周志华. 机器学习.

[4] 邱锡鹏. 神经网络与深度学习.

[5] 李航. 统计学习方法.

# A 西瓜数据集

| 编号 | 色泽 | 根蒂 | 敲声 | 纹理 | 脐部 | 触感 | 密度 | 含糖率 | 好瓜 |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 青绿 | 蜷缩 | 浊响 | 清晰 | 凹陷 | 硬滑 | 0.697 | 0.46 | 是 |
| 2 | 乌黑 | 蜷缩 | 沉闷 | 清晰 | 凹陷 | 硬滑 | 0.774 | 0.376 | 是 |
| 3 | 乌黑 | 蜷缩 | 浊响 | 清晰 | 凹陷 | 硬滑 | 0.634 | 0.264 | 是 |
| 4 | 青绿 | 蜷缩 | 沉闷 | 清晰 | 凹陷 | 硬滑 | 0.608 | 0.318 | 是 |
| 5 | 浅白 | 蜷缩 | 浊响 | 清晰 | 凹陷 | 硬滑 | 0.556 | 0.215 | 是 |
| 6 | 青绿 | 稍蜷 | 浊响 | 清晰 | 稍凹 | 软粘 | 0.403 | 0.237 | 是 |
| 7 | 乌黑 | 稍蜷 | 浊响 | 稍糊 | 稍凹 | 软粘 | 0.481 | 0.149 | 是 |
| 8 | 乌黑 | 稍蜷 | 浊响 | 清晰 | 稍凹 | 硬滑 | 0.437 | 0.211 | 是 |
| 9 | 乌黑 | 稍蜷 | 沉闷 | 稍糊 | 稍凹 | 硬滑 | 0.666 | 0.091 | 否 |
| 10 | 青绿 | 硬挺 | 清脆 | 清晰 | 平坦 | 软粘 | 0.243 | 0.267 | 否 |
| 11 | 浅白 | 硬挺 | 清脆 | 模糊 | 平坦 | 硬滑 | 0.245 | 0.057 | 否 |
| 12 | 浅白 | 蜷缩 | 浊响 | 模糊 | 平坦 | 软粘 | 0.343 | 0.099 | 否 |
| 13 | 青绿 | 稍蜷 | 浊响 | 稍糊 | 凹陷 | 硬滑 | 0.639 | 0.161 | 否 |
| 14 | 浅白 | 稍蜷 | 沉闷 | 稍糊 | 凹陷 | 硬滑 | 0.657 | 0.198 | 否 |
| 15 | 乌黑 | 稍蜷 | 浊响 | 清晰 | 稍凹 | 软粘 | 0.36 | 0.37 | 否 |
| 16 | 浅白 | 蜷缩 | 浊响 | 模糊 | 平坦 | 硬滑 | 0.593 | 0.042 | 否 |
| 17 | 青绿 | 蜷缩 | 沉闷 | 稍糊 | 稍凹 | 硬滑 | 0.719 | 0.103 | 否 |

# B 对率回归

```
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
from sklearn import linear_model

def sigmoid(x):
    """
    计算sigmoid函数的值。

    :param x: 输入值，可以是标量、向量或矩阵
    :return: sigmoid函数的结果，范围在(0, 1)之间
    """
    return 1 / (1 + np.exp(-x))

class LogisticRegressionModel:
    def __init__(self, learning_rate=1.2, num_iterations=100,
        print_cost=False):
        """
        初始化对率回归模型。

        :param learning_rate: 梯度下降法的学习率（步长）
        :param num_iterations: 迭代次数
        :param print_cost: 是否在每100次迭代后打印当前的损失函数
            值
        """
        self.learning_rate = learning_rate  # 学习率决定每次参数
            更新的步长
        self.num_iterations = num_iterations  # 最大迭代次数
        self.print_cost = print_cost  # 控制是否打印每次迭代的损
            失值
        self.beta = None  # 模型的参数（权重），将通过训练学习到
```

```
28
29      def J_cost(self, X, y, beta):
30          """
31          计算对率回归的损失函数（交叉熵损失）。
32
33          :param X: 特征矩阵，shape为（n_samples, n_features）
34          :param y: 标签向量，shape为（n_samples,）
35          :param beta: 参数向量，shape为（n_features + 1, 1）
36          :return: 当前参数下的损失函数值
37          """
38          # 添加偏置列（全为1的列）到X中
39          X_hat = np.c_[X, np.ones((X.shape[0], 1))]
40          beta = beta.reshape(-1, 1)
41          y = y.reshape(-1, 1)
42
43          # 计算损失函数（负对数似然）
44          Lbeta = -y * np.dot(X_hat, beta) + np.log(1 + np.exp(np.
                dot(X_hat, beta)))
45          return Lbeta.sum()
46
47      def gradient(self, X, y, beta):
48          """
49          计算损失函数对参数beta的梯度。
50
51          :param X: 特征矩阵，shape为（n_samples, n_features）
52          :param y: 标签向量，shape为（n_samples,）
53          :param beta: 参数向量，shape为（n_features + 1, 1）
54          :return: 损失函数的梯度，shape为（n_features + 1, 1）
55          """
56          # 添加偏置列（全为1的列）到X中
57          X_hat = np.c_[X, np.ones((X.shape[0], 1))]
58          beta = beta.reshape(-1, 1)
59          y = y.reshape(-1, 1)
```

```
60
61          # 计算预测概率
62          p1 = sigmoid(np.dot(X_hat, beta))
63
64          # 计算梯度
65          grad = (-X_hat * (y - p1)).sum(0)
66          return grad.reshape(-1, 1)
67
68      def update_parameters(self, X, y, beta):
69          """
70          使用梯度下降法更新参数。
71
72          :param X: 特征矩阵, shape为 (n_samples, n_features)
73          :param y: 标签向量, shape为 (n_samples,)
74          :param beta: 初始的参数向量, shape为 (n_features + 1, 1)
75          :return: 更新后的参数向量, shape为 (n_features + 1, 1)
76          """
77          for i in range(self.num_iterations):
78              # 计算当前梯度
79              grad = self.gradient(X, y, beta)
80
81              # 更新参数
82              beta = beta - self.learning_rate * grad
83
84              # 打印每100次迭代的损失值
85              if (i % 100 == 0) and self.print_cost:
86                  print(f'{i}th iteration, cost is {self.J_cost(X,
                        y, beta)}')
87
88          return beta
89
90      def initialize_beta(self, n):
91          """
```

```
92          随机初始化参数beta。
93
94          :param n: 特征的数量
95          :return: 随机初始化的参数向量，shape为 (n_features + 1,
                1)
96          """
97          # beta的初始化为n+1个参数，其中n是特征数量，1是偏置项
98          return np.random.randn(n + 1, 1) * 0.5 + 1
99
100     def fit(self, X, y):
101         """
102         训练对率回归模型，使用梯度下降法学习参数。
103
104         :param X: 特征矩阵，shape为 (n_samples, n_features)
105         :param y: 标签向量，shape为 (n_samples,)
106         """
107         m, n = X.shape  # m为样本数量，n为特征数量
108
109         # 初始化参数
110         self.beta = self.initialize_beta(n)
111
112         # 使用梯度下降更新参数
113         self.beta = self.update_parameters(X, y, self.beta)
114
115     def predict(self, X):
116         """
117         对新数据进行预测，返回二分类结果（0或1）。
118
119         :param X: 特征矩阵，shape为 (n_samples, n_features)
120         :return: 预测结果，shape为 (n_samples,)。1表示正类，0表
                示负类。
121         """
122         # 添加偏置列
```

```
123         X_hat = np.c_[X, np.ones((X.shape[0], 1))]
124
125         # 计算预测概率
126         p1 = sigmoid(np.dot(X_hat, self.beta))
127
128         # 根据概率阈值0.5进行分类
129         return (p1 >= 0.5).astype(int)
130
131 if __name__ == '__main__':
132     # 读取数据
133     data_path = r'.\watermelon3_0_Ch.csv'
134     data = pd.read_csv(data_path).values
135
136     # 处理标签
137     is_good = data[:, 9] == '是'
138     is_bad = data[:, 9] == '否'
139
140     # 选择特征和标签
141     X = data[:, 7:9].astype(float)
142     y = data[:, 9]
143
144     # 将标签转换为0和1
145     y[y == '是'] = 1
146     y[y == '否'] = 0
147     y = y.astype(int)
148
149     # 绘制数据分布
150     plt.scatter(data[:, 7][is_good], data[:, 8][is_good], c='k',
            marker='o')
151     plt.scatter(data[:, 7][is_bad], data[:, 8][is_bad], c='r',
           marker='x')
152     plt.xlabel('density')
153     plt.ylabel('sugar')
```

```
154
155     # 自定义模型的训练与绘图
156     model = LogisticRegressionModel(learning_rate=0.3,
            num_iterations=1000, print_cost=True)
157     model.fit(X, y)
158     beta = model.beta
159     w1, w2, intercept = beta
160     x1 = np.linspace(0, 1)
161     y1 = -(w1 * x1 + intercept) / w2
162     ax1, = plt.plot(x1, y1, 'b-', label='My Logistic (gradDesc)'
            , linewidth=3)
163
164     # sklearn模型的训练与绘图
165     lr = linear_model.LogisticRegression(solver='lbfgs', C=1000)
166     lr.fit(X, y)
167     w1_sk, w2_sk = lr.coef_[0, :]
168     x2 = np.linspace(0, 1)
169     y2 = -(w1_sk * x2 + lr.intercept_) / w2_sk
170     ax2, = plt.plot(x2, y2, 'r--', label='sklearn Logistic',
            linewidth=3)
171
172     # 显示图例
173     plt.legend(loc='upper right')
174     plt.show()
```

# C 感知机

```python
import numpy as np

# 感知机类
class Perceptron:
    def __init__(self, input_size, lr=0.1, epochs=10):
        # 初始化权重为0
        self.weights = np.zeros(input_size + 1)  # +1 是为了 bias
        self.lr = lr  # 学习率
        self.epochs = epochs  # 迭代次数

    def activation_fn(self, x):
        # 阈值函数（单位阶跃函数）将{+1, -1}映射到{1, 0} 方便AND和OR计算
        return 1 if x >= 0 else 0

    def predict(self, x):
        # 计算加权和
        z = np.dot(x, self.weights[1:]) + self.weights[0]
        # 应用激活函数
        return self.activation_fn(z)

    def fit(self, X, d):
        for _ in range(self.epochs):
            for i in range(len(d)):
                # 预测输出
                x_i = X[i]
                y = self.predict(x_i)
                # 计算误差
                error = d[i] - y
                # 更新权重和偏置项
```

```
30                 self.weights[1:] += self.lr * error * x_i
31                 self.weights[0] += self.lr * error
32
33 if __name__ == '__main__':
34     # 定义输入数据 (AND, OR)
35     X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
36
37     # AND 操作的目标输出
38     d_and = np.array([0, 0, 0, 1])
39     # OR 操作的目标输出
40     d_or = np.array([0, 1, 1, 1])
41
42     # 创建感知机实例，分别训练 AND 和 OR
43     print("Training for AND operation")
44     perceptron_and = Perceptron(input_size=2)
45     perceptron_and.fit(X, d_and)
46     print("\nTraining for OR operation")
47     perceptron_or = Perceptron(input_size=2)
48     perceptron_or.fit(X, d_or)
49
50     # 测试感知机的输出
51     output_and = [perceptron_and.predict(x) for x in X]
52     output_or = [perceptron_or.predict(x) for x in X]
53
54     # 打印输入和对应的输出
55     print("\nFinal Outputs:")
56     print("AND Operation:")
57     for i, x in enumerate(X):
58         print(f"Input: {x}, Predicted Output: {output_and[i]}")
59     print("\nOR Operation:")
60     for i, x in enumerate(X):
61         print(f"Input: {x}, Predicted Output: {output_or[i]}")
```